CS5707 Artificial Intelligence
Lab 2. Search

Yongmin Li*

We will use the Google Colaboratory (or Google Colab or simply Colab) in the labs of this module, which is a free Jupyter notebook environment hosted by the Google cloud services. The Colab uses the same format of Jupyter Notebook, a popular interactive development platform for Julia, Python and R.

Aims and Objectives

In this lab, we will implement the search algorithms we have studied to solve some real-world problems.

1. The first is the route planning problem we have used to introduce the search problem.
2. The second is the 8-puzzle problem I have left you for private study.
3. The last one is a maze solver (to have some fun).

Many people would feel quite challenging to solve these problems by hands. Now you can show that, only within one week's study in Artificial Intelligence, you could do it with a simple program!

After this session, you should achieve the following objectives:

1. Formulating a search problem by specifying the states, actions, goal test and path cost;
2. Understanding the process of various search algorithms including A*, breadth-first, depth-first, uniform-cost, greedy, etc.;
3. Designing and implementing a software solution to a search problem;
4. Evaluating the performance of these different search algorithm.

Time Required

2 hours (with supervision).

Please note that the above number is only indicative and should be regarded as an average amount of time for all students. You may require significantly more or less time depending on personal circumstances, e.g. knowledge and understanding to the relevant subject topics, experience of the software used, and level of programming skills. Please do prepare to use more time outside the officially arranged sessions.

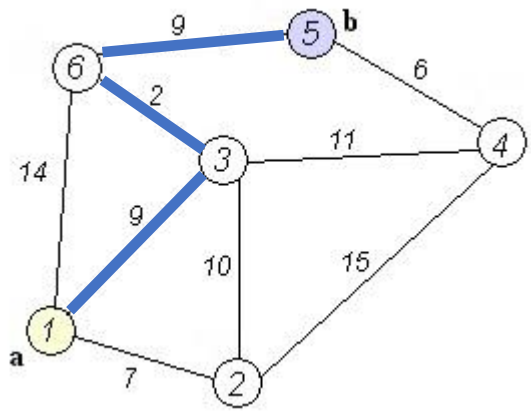
---

* First draft September 2020. These materials have been used for "CS5707 Artificial Intelligence" at Department of Compute Science, Brunel University London.
* Revised October 2021.

Task 2.1 Route Planning

The figure below shows the problem we used in the lecture. There are 6 cities numbered from 1 to 6. The distances between them, if connected directly, are show on the edges in the graph.

Your task is to write a program to find a route, ideally an optimal one (shortest in this case), from any given pair of cities. For example, if we start from city 1 and the destination is 5, the optimal route is highlighted in the graph with a total cost of 20.



We will use the Python package simpleai. The project website is at https://pypi.org/project/simpleai/

The online documentation of simpleai can be found at https://simpleai.readthedocs.io/en/latest/

All the search algorithms we discussed in the lecture have been provided by simpleai including breadth-first, depth-first, uniform-cost, greedy and A* search algorithms. With simpleai, you can use these algorithms directly instead of going through the hard process of implementing them from scratch. However, you still need to define your problem properly.

In the lecture, we discussed the four important items for the definition of a search problem: state, action, goal test and path cost. In the route planning problem, we will formulate the problem as follows.

1. State: number 1 to 6 for the cities
2. Action: we just use the new state for simplicity of implementation. For example, if the current state is 1 and the next state is 3, we just use 3 as action – meaning travelling from 1 to 3.
3. Goal test: check if the current state is the destination, e.g. 5.
4. Path cost: This is given is the graph. We transform it into a 6x6 array whose elements are the distance between a pair of cities (0 for self and 'inf' for unconnected).

```
COSTS = [
    [0, 7, 9, 'inf', 'inf', 14],
```

```
    [7, 0, 10, 15, 'inf', 'inf'],
    [9, 10, 0, 11, 'inf', 2],
    ['inf', 15, 11, 0, 6, 'inf'],
    ['inf', 'inf', 'inf', 6, 0, 9],
    [14, 'inf', 2, 'inf', 9, 0]
]
```

The base line implementation is provided in “Search_Route.ipynb” if you are struggling to start with. First install the simpleai package into your Colab environment by calling “pip install”, then import the packages we need.

```
!pip install simpleai
from simpleai.search import SearchProblem, astar, greedy, breadth_first, de
pth_first, uniform_cost
import math
```

Now we create a new class “Route” from “SearchProblem” provided in the package. The functions inside the newly created class are all based on the problem formulation we discussed above.

The initialisation function takes inputs of the start and end states. Because the numbers start from 1 in the graph but index in Python starts from 0, so we do -1 there.

The function “actions” checks all the directly reachable states from the current state, and appends them into the list to be returned. The function “result” will update the state after an action. In our case, it is just the action itself. The function “is_goal” checks if the current state is the goal. Finally the function “cost” return the cost of an action.

The above four functions are required in order to use the simpleai package. Read the online documentation for more details of this.

```
class Route(SearchProblem):

    # initialise with the initial and goal states.
    def __init__(self, initial, goal):
        self.initial = initial-1
        self.goal = goal-1
        super(Route, self).__init__(initial_state=self.initial)

    # add all connected nodes, i.e. those with a cost of not 0 or inf
    def actions(self, state):
        actions = []
        for action in range(len(COSTS[state])):
            if COSTS[state][action] not in ['inf', 0]:
                actions.append(action)
        return actions
```

```
    # the result state is just the action as defined
    def result(self, state, action):
        return action

    # check if goal is reached
    def is_goal(self, state):
        return state == self.goal

    # return the cost between two states
    def cost(self, state, action, state2):
        return COSTS[state][action]
```

Once the problem is defined, to use a search algorithm to find a route is quite straightforward. In the following example, we use the uniform-cost search to find the route from 1 to 5.

```
problem = Route(1, 5)
result = uniform_cost(problem)
path = [x[1]+1 for x in result.path()]
print("The route is %s, and total cost is %s" %(path, result.cost))
```

Hopefully you will produce the result like:

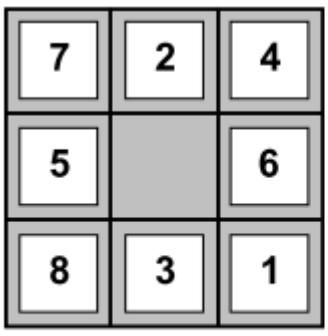
```
The route is [1, 3, 6, 5], and total cost is 20.
```

Requirement for lab report:

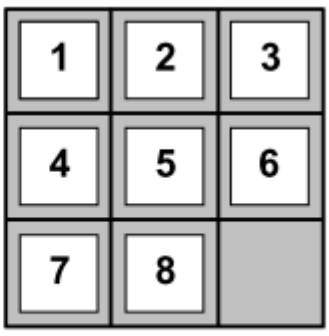
1. Use three methods of uniform-cost, breadth-first and depth-first search to find route from 1 to 5.
2. Compare the results from these methods, and analyse their performance.

Task 2.2 8-Puzzle Problem

The 8-puzzle problem is described in details in (Russell and Norvig 2021). In this task, we will develop a solution to this problem, again, using the simpleai package.



Start State



Goal State

Let us first define the problem.

1. State: a 3x3 array with numbers 1-8 for the tiles and ‘E’ for the blank space.
2. Action: instead of moving the tiles, it would be much easier to implement if we move the blank space up, down, left and right. We need to check the boundary conditions to decide which of the four moves are valid for a given state.
3. Goal test: check if the current state is the goal state, such as shown in the above figure.
4. Path cost: equal step cost – assigned to 1 per move.

A base line implementation is provided in “Search_8Puzzel.ipynb”, which was adapted from an example program in the simpleai package. For visualisation purpose, the states of the problem are given in a string. So some small conversion functions between string and array are included in the code.

The key to the success of informed (heuristic) search is the heuristic function. Two heuristic functions are discussed in the textbook.

h1: the number of misplaced tiles
h2: the sum of distances of tiles from their target positions

The solution depth for the initial state and goal state in the above figure is 26. If an appropriate method is used, for example, A* search with a good heuristic function, the program should be able to find the solution rather quickly. However, it may take much longer or even crash the Google Colab engine if inappropriate methods are used!

It is highly recommended that you read carefully the comparison of performance of various search methods in (Russell and Norvig 2021) pp116-117, and make sure you understand that, even for such a seemingly simple problem, the number of nodes generated in the search can go beyond control quickly if a poor method or a poor heuristic function is chosen.

Complete the program, and test it on the following two initial states (one simple case with solution depth of 4, and a difficult one with solution depth of 26).

Simple case:

```
# Starting point - solution depth = 4
# INITIAL = '''\
# 1-4-2
# 3-5-8
# 6-7-E'''
```

Difficult case:

```
# Starting point - solution depth = 26
# INITIAL = '''\
# 7-2-4
# 5-E-6
# 8-3-1'''
```

Requirement for lab report:

1. Use various methods such as A*, greedy, breadth-first and depth-first search to the simple case, compare the results and discuss the performance.
2. Implement both h1 and h2 as heuristic function, use them with greedy and A* search to solve the difficult case (solution depth = 26). Discuss your results.

Task 2.3 Maze Solver

In this task, we will write a program to solve a maze problem, i.e. find a route (ideally the shortest one) from the start (labelled as 'O' in the map) to the target (labelled as 'X'). Other symbols in the map are '#' for walls that you cannot go through and blanks that you can go through freely.

```
MAP = """
##############################
#          #                 #   #
# ####     ########          #   #
#  o #     #                 #   #
#    ###      ####    ######    #
#          ####        #         #
#             #  #     #    #### #
#      ######     #         # x   #
#         #       #               #
##############################
"""
```

Hopefully after the previous exercises, you would feel comfortable to define this problem. A few hints:

1. You may consider to use the location (x, y) in the map as the state.
2. The actions for a given state depend on the surroundings. For example, if there is a wall to the right, you cannot move to the right.
3. Let us start with a simple case where only four moves are allowed – up, down, left and right. You may use equal cost for each move.

Again, a base line implementation is provided in "Search_Maze.ipynb" that was adapted from an example program in simpleai.

Requirement for lab report:

1. Define the search problem by briefly describing the state, action, goal test and path cost.
2. Define a heuristic function for the search problem.
3. Use A* to find a route from start to target, displaying number of moves and total costs.
4. Use other search algorithms such as greedy, breadth-first and depth-first etc. Check if you can find a route as good as the one from A*. Explain your results.

5. Extend the search by four additional diagonal moves – up left, up right, down left and down right. Assign these diagonal moves with 1.4x of the other moves. Try the A* algorithm again and compare the results.